

Lecture 07c.

Trees : Properties

Trees: Height vs size

- Height is a vital property of trees
- Tree algorithms often work by going down the tree level by level – following a path from root to a leaf – or vice versa
 - Running times depend on the number of levels, rather than the size (total number of nodes)
- Hence, **running times are often $O(\text{height})$**
- But we usually only know the number of nodes, n , and so need to relate the height, h , with n .

Height h vs Number of nodes n

- We want to put limits on the relationships between
 - Tree height h
 - Number of nodes n
- E.g. for a given number of nodes what range can the height have?
- Which trees are the extreme cases, etc.

Maximum h for a given n

- For a given number of nodes n , the maximum height is clearly when all the nodes have at most one child, and we get a long chain.
- Essentially, this is the special case of a tree that is a linked list
 - Then the number of edges to traverse will just be $n-1$
 - So maximum height $h_{\max}(n) = n-1$
 - Hence, $h(n)$ is $O(n)$

Minimum h for a given n ?

- In the extreme case, the arity could be $n-1$ – that is, one node is the parent and the others are children of the parent.
 - This gives a tree of height $h = 1$
 - However, it is too special case a case to be useful.
 - The list of children would just be a list and so the tree would not be very useful
 - Hence, look at a fixed arity.
 - Most useful is to look at binary trees

Binary Tree: Min h for given n ?

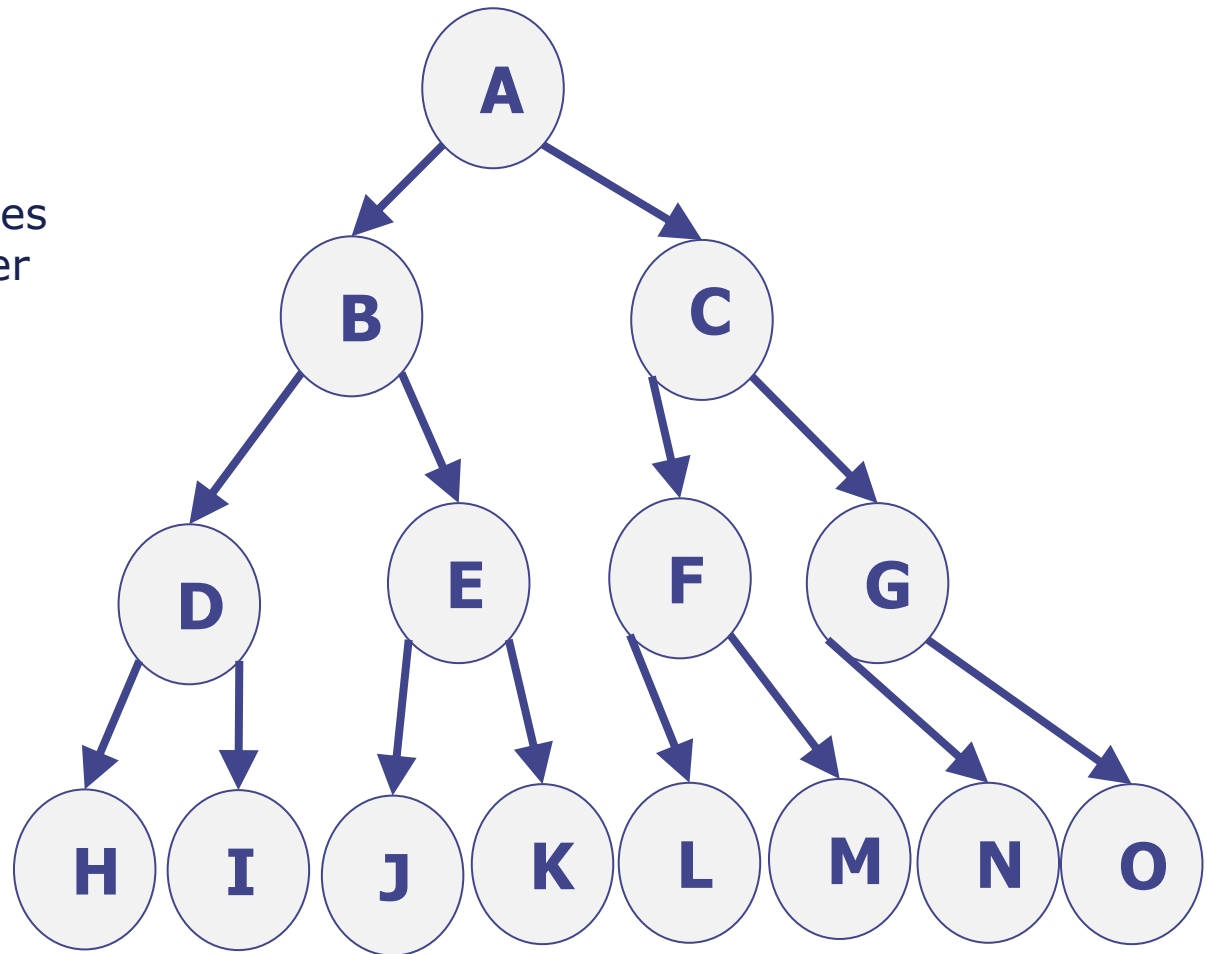
- Easiest is to first flip the question around:
- What is the largest n for a binary tree with a given h
- This clearly happens when the tree is totally full down to the level h
 - There is no more space to place any more nodes without increasing h

Perfect Binary Tree : Node Counts

For levels 0,1,2,3 count the

- Number of nodes at precisely that level
- Total number of nodes at that level or higher in the tree

Level L	At L	Total for $\leq L$
0	1	1
1	2	3
2	4	7
3	8	15



Perfect Binary Tree : General

Level L	At L	Total for $\leq L$
0	1	1
1	2	3
2	4	7
3	8	15

Strongly suggests

- Number of nodes at precisely level L is 2^L
- Total number of nodes at level less (i.e. higher in the tree) is $2^{(L+1)} - 1$

Perfect Binary Tree: Numbers of nodes is exponential in the depth

Can prove these by induction:

Perfect Binary: Nodes at a Level

Claim

- Number of nodes $M(L)$ at precisely level L is 2^L

Proof by induction:

Base case: $L=0$ the only node is the parent, so $M(0)=1 = 2^0$ ok

Step case: Assume is true at level L : $M(L) = 2^L$ – induction hypothesis

At level $L+1$ we will have precisely twice the number of nodes, as each node has precisely two children at the next level

So, $M(L+1) = 2 M(L)$

$= 2 \cdot 2^L$ (using the induction hypothesis)

$= 2^{(L+1)}$ matching what is expect at level $L+1$

Proved.

Perfect Binary: Nodes at a Level or above

Claim:

- Number of nodes $n(L)$ at precisely level L or less is $2^{(L+1)} - 1$

Directly do the geometric sum:

$$n(L) = M(0) + M(1) + M(2) + \dots + M(L) = 1 + 2 + 4 + 2^L$$

Proof by induction:

Base case: $L=0$ the only node is the parent, so $n(0)=1$ which matches $2^{(0+1)}-1$

Step case: Assume claim is true at level L .

At level $L+1$ we will add $M(L+1)$ nodes for that next level

So, $n(L+1) = n(L) + M(L+1)$

$$= 2^{(L+1)} - 1 + 2^{(L+1)} \quad (\text{using the induction hypothesis})$$

$$= 2 \cdot 2^{(L+1)} - 1$$

$$= 2^{((L+1)+1)} - 1 \text{ matching what is expect at level } L+1$$

What is the height of a (perfect) binary tree of size n (with n nodes)?

We know that $n = 2^{h+1} - 1$.

So, $2^{h+1} = n + 1$.

$h + 1 = \log_2(n+1)$

$h = \log_2(n+1) - 1$.

So, the height of the tree is logarithmic in the size of the tree.

The size of the tree is exponential in the height (number of levels) of the tree.

What is the height of an arbitrary binary tree of size n (with n nodes)?

- If the tree is perfect, then it has height that is logarithmic in the size of the tree: $\Theta(\log(n))$
- If it is imperfect, then for the same n it must have at least this height: $\Omega(\log(n))$
- However, consider a simple “chain” – basically a linked list
 - each non-leaf node has just one child. It is still a binary tree – just a special case.
 - It has height $n-1$ and this is “obviously” maximal height
 - Hence, trees have height $O(n)$.
- **Hence, for a general binary tree on n nodes, the height is $\Omega(\log(n))$ and $O(n)$**

Minimum Expectations

- Definitions associated with trees
- **Post- Pre- and In-order traversal and their usages**
- Implementation methods
 - nodes
 - array based
- Binary Trees – meaning of proper, perfect
- **Sizes and heights of binary trees**